ReduLink: Context-Bound Reference Substitution over Encrypted QUIC Streams

Michél Nguyen
Computer Science, University of the People
595 E. Colorado Boulevard, Suite 623, Pasadena, CA 91101, USA
ORCID: 0000-0001-6834-4422 | Corresponding author: minhmichel@outlook.de

Abstract

Endpoint redundancy elimination predates QUIC, but encrypted transports make its placement and state discipline newly important. ReduLink is an application-stream representation for cooperating QUIC endpoints that replaces a repeated chunk with a reference to preprovisioned receiver state. Its contribution is not a new matching algorithm. It is a concrete QUIC mapping with canonical, context-bound record commitments, bounded true-LRU state, declared reconstruction limits, and batched miss repair. QUIC/TLS remains the network-security boundary; the record HMAC is a defensive commitment against stale-state and cross-context confusion inside an endpoint. We implement the mapping with aioquic and evaluate only byte accounting and exact reconstruction, excluding diagnostic messages and avoiding latency or fairness claims. On three hash-pinned public release pairs transferred as named objects, a binary HMAC-frame profile under a deterministic public artifact key reconstructs exactly at 1.81x, 7.48x, and 3.92x. A whole-object content-addressed baseline reaches 1.65x, 5.52x, and 3.16x, while a prior-stream zstd dictionary wins on two pairs and loses on one. Real rsync dominates the corresponding raw source-tree updates. In native QUIC tests, a 98,304-byte warm update uses 30,851 protocol-stream bytes after repair, or 3.19x, whereas an independent compressed control expands to 0.91x. With matched sender and receiver budgets, an 8 MiB run reaches 10.06x; a 16 MiB run falls below break-even at 8,192 chunks and recovers to 10.10x at 24,576 chunks. These results delimit ReduLink as a scoped object-stream mechanism, not a universal compressor or a replacement for rsync or HTTP dictionary transport.

Keywords: QUIC; redundancy elimination; deduplication; content addressing; shared dictionaries; reproducible systems evaluation

1. Introduction

Encryption does not remove redundancy, but it removes the plaintext vantage used by transparent network redundancy elimination. Classic link and network-wide systems identify repeated traffic inside the network [1-3]. QUIC instead combines encrypted streams, loss recovery, and congestion control between endpoints [14-18]. Any redundancy suppression that preserves end-to-end protection must therefore occur before encryption at the sender and after decryption at the receiver.

Endpoint placement is not itself novel. EndRE explicitly moved redundancy elimination to end systems and could process traffic before SSL or other application encryption [4]. DOT offered hash-named chunks, caching, and explicit transfer misses as an application data-transfer service [5]. PACK and CoRE developed receiver-driven or cooperative endpoint mechanisms for cloud traffic [6,7]. ReduLink starts from this lineage. The research question is narrower: how should a warm-state reference be represented, bounded, and checked when it is carried inside a modern encrypted QUIC stream, and where does that representation remain competitive with simpler tools?

Mechanism choice depends on the workload. A file-tree update is better matched to rsync or a related delta protocol [8,9]. An HTTP response with an eligible prior response is better matched to Compression Dictionary Transport (CDT) [11-13]. A locally repetitive object may need only ordinary compression. ReduLink targets a different serving abstraction: an endpoint already has manifest-agreed object or chunk state, but the sender needs explicit, individually checked reconstruction steps and a deterministic repair path without abandoning QUIC streams.

The paper asks three questions:

- RQ1: Can a QUIC application-stream mapping make warm-state references explicit, context-bound, bounded, and fail closed without a custom QUIC extension?
- RQ2: On which public object-update workloads does the mapping reduce bytes, and when do rsync, whole-object content addressing, chunk tokens, gzip, or zstd dictionaries perform better?
- RQ3: How do matched endpoint capacity and semantic reference misses change protocol-stream bytes and the break-even point?

The main contribution is a reproducible answer to these questions. The artifact provides a binary mapping, an implemented receive-state machine, exact reconstruction checks, same-layer directional accounting, hash-pinned corpora, explicit negative controls, and tests for malformed context encoding, stale dictionary state, replay, length expansion, and quota violations [27,28]. The paper intentionally does not infer WAN latency, congestion fairness, or production throughput from single-host experiments.

2. Related Work and Novelty Boundary

2.1 Network and endpoint redundancy elimination

Spring and Wetherall established protocol-independent repeated-byte suppression on a link [1]. SmartRE coordinated caches across a network, and later measurements quantified repeated traffic at broader vantage points [2,3]. Those designs motivate the byte-saving objective, but their network placement is incompatible with end-to-end encrypted payload inspection.

EndRE is the closest conceptual predecessor because it places redundancy elimination at end systems and explicitly benefits encrypted traffic [4]. DOT is also close: applications submit data to a transfer service that uses content-based names and can retrieve missing chunks [5]. PACK shifts computation to clients through receiver predictions and predictive acknowledgements [6]. CoRE coordinates endpoint state to reduce cloud bandwidth cost [7]. ReduLink does not claim to invent endpoint substitution, hash-named chunks, or miss recovery. Its distinct scope is the composition of a QUIC stream mapping with canonical per-connection context, bounded receive invariants, an executable binary accounting profile, and evidence that directly includes its closest baselines.

2.2 Delta transfer, content addressing, and compression

The rsync algorithm identifies matching regions at a remote receiver using rolling and strong checksums [8]. LBFS applies content-defined chunks to a file system over constrained links [9]. REBL combines chunk suppression, resemblance detection, and delta encoding [10]. These systems show why a fixed-chunk reference layer cannot be evaluated only against raw transfer. Our evaluation therefore includes real rsync, a 4 KiB token baseline, whole-object content addressing, gzip, and a verified zstd prior-stream dictionary.

HTTP delta encoding and VCDIFF standardize response differencing [11,12]. RFC 9842 standardizes CDT, including dictionary advertisement, SHA-256 identity, same-origin and readability constraints, dictionary-aware Brotli or Zstandard encodings, and failure handling [13]. ReduLink is not a stronger replacement for CDT. CDT is preferable for HTTP content coding. ReduLink exposes chunk-level resolution and batched repair outside HTTP, which may fit object-stream protocols that already manage warm state.

Tentackle's TRIP-over-QUIC module is a close industrial design: it carries an existing remote-invocation protocol over QUIC streams while reusing TRIP serialization, dictionary, registry, and back-reference machinery; its compressed scheme adds streaming deflate [30]. ReduLink differs by specifying authenticated, context-bound chunk records, bounded true-LRU admission, an explicit batched miss-repair exchange, and byte-accounted comparison against transfer and dictionary baselines. The Tentackle documentation is implementation prior art rather than a peer-reviewed performance study, so no comparative performance claim is inferred from it.

Content-defined chunking can improve resilience to insertions, and FastCDC shows how boundary selection can be accelerated [22]. The artifact retains a simple rolling-hash CDC prototype for exploratory use, but the main results use fixed 4 KiB or 1 KiB chunks. No FastCDC performance or accuracy claim is made. This choice prevents an unvalidated chunker from carrying the paper's conclusions.

Table 1. Closest-work matrix and the narrow ReduLink novelty claim.

Approach	Endpoint abstraction	Reuse unit	Miss or recovery path	Context and transport focus
EndRE [4]	Endpoint RE service before encryption	Chunks	Endpoint synchronization	Encrypted traffic support; predates QUIC
DOT [5]	Application transfer service	Hash-named chunks	Explicit cache misses	Content naming and pluggable transfer
PACK [6]	Receiver-driven endpoint TRE	Predicted chunks	PRED-ACK path	TCP/cloud cost and client computation
CoRE [7]	Cooperative endpoint TRE	Chunks	Cooperative state	Cloud bandwidth cost
rsync/LBFS [8,9]	File or file-system delta	Rolling blocks or CDC	Delta negotiation	Named files and file trees
HTTP CDT [11-13]	HTTP content coding	External codec dictionary	Response failure	HTTPS origin, availability, and dictionary identity
Tentackle TRIP/QUIC [30]	Remote invocation over QUIC streams	TRIP dictionary/back-references	Existing TRIP behavior	QUIC adapter; optional deflate mode
ReduLink	QUIC application stream	FULL/REF records	Batched MISSING and FULL repair	Canonical connection/stream context and bounded reconstruction

3. System Model and Protocol

3.1 Deployment assumptions

The sender and receiver are cooperating endpoints of one server-authenticated QUIC connection. Both can access plaintext inside their endpoint process. Application-level client authorization is assumed by the deployment and is not implemented by the artifact. The receiver has a warm dictionary that was provisioned out of band or agreed by a signed or hash-pinned manifest. The current protocol does not discover, advertise, or admit dictionaries on the wire. This assumption is appropriate for a registry client, backup agent, managed CDN endpoint, or enterprise service with explicit state management. It is not appropriate for arbitrary cross-origin or cross-user reuse.

Dictionary scope is part of policy. A conservative deployment uses per-connection or per-origin state. Per-tenant state is possible only when authorization and data-classification policy permit it. Global cross-user deduplication is excluded because deduplication outcomes and compressed lengths can reveal content existence [23,24]. RFC 9842 applies analogous origin, readability, and storage-partitioning precautions to HTTP dictionaries [13].

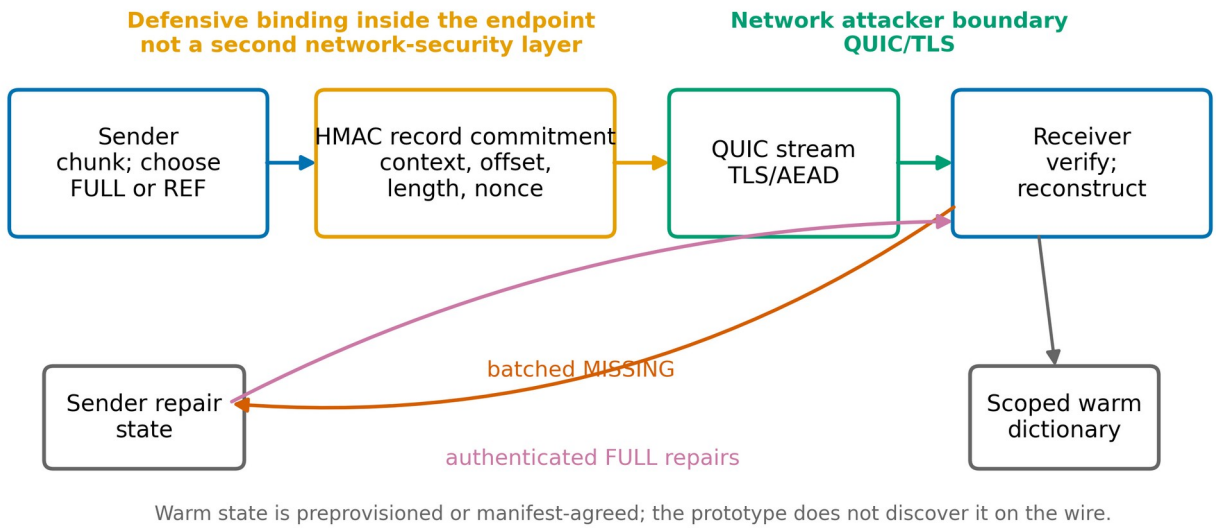

Figure 1. ReduLink mapping and trust boundaries. QUIC/TLS protects the network path. HMAC records defensively bind decoded representation state inside the endpoints.

3.2 Binary message sequence

ReduLink is implemented as an application codec carried in one QUIC bidirectional stream. It is not a custom QUIC frame. The first client-initiated stream ID is read from aioquic and used in key context and record verification. Object-suite experiments use a separate 62-bit logical object-context identifier derived from the object name; the suite checks for collisions and serializes every secure frame through the binary encoder.

Table 2. Implemented message sequence. Diagnostic STATS bytes are not protocol evidence.

Step	Direction	Message	Implemented purpose
1	Client to server	HELLO	Version, chunk size, frame count, declared input length, and SHA-256 digest
2	Client to server	FULL or REF	Initial ordered records with sequence and reconstructed offset
3	Client to server	END_ROUND	Close the initial record round
4	Server to client	MISSING	One batched list of missing sequence, chunk identifier, and length
5	Client to server	FULL repair	Authenticated literals after sender validation of every MISSING item
6	Client to server	FINISH	Request final sequence, length, and digest validation
7	Server to client	STATS	Experiment diagnostics only; excluded from the protocol multiplier

Repair is batched after END_ROUND, not immediate. There is no DICT_ACK in the implemented profile. A sender rejects an out-of-range, duplicate, non-REF, wrong-identifier, or wrong-length repair request. A receiver rejects a repair that does not match its pending sequence, identifier, length, and offset. These checks make the recorded state transition unambiguous, but they do not optimize time to first reconstructed byte. HELLO declares the exact frame count, and the implementation rejects a count that could not fit in one MISSING message under the 16 MiB message cap.

3.3 Records, state, and wire cost

A FULL record contains the chunk bytes. A REF record contains no chunk payload and resolves through the receiver dictionary. Both carry an epoch, scope, logical stream context, reconstructed offset, length, nonce, 128-bit keyed chunk identifier, and 128-bit record tag. Successful REF hits refresh true-LRU recency at both sender and receiver. A FULL insert also refreshes the entry and evicts the least-recently used entry when the configured chunk count is exceeded.

The binary frame has 85 fixed bytes including its length prefix and message type, plus the UTF-8 scope. The native 16-byte scope therefore costs 101 bytes per FULL or REF before a FULL payload. The 27-byte object-suite scope costs 112 bytes. Earlier 24-byte and 32-byte model prices are retained only as exploratory model outputs and are not used for headline conclusions. QUIC packet headers, ACKs, and link-layer overhead are outside the stream-payload metric.

Before accepting data, the receiver checks the QUIC/TLS-protected HELLO declaration against server-configured expected values and ensures that the declared input length does not exceed a configured reconstruction quota. Every record length must be positive and no larger than the negotiated fixed chunk size. Reconstructed offsets are restricted to the QUIC 62-bit range. FINISH succeeds only if all expected sequence numbers are present and the reconstructed length and SHA-256 digest equal HELLO.

4. Security and Privacy Analysis

4.1 Attacker boundary

QUIC uses TLS to protect application streams, and QUIC loss recovery and congestion control operate on encrypted transport data [14-16]. The on-path attacker is therefore handled by QUIC/TLS, not by ReduLink. TLS 1.3 exporters derive application keying material from an exporter master secret and explicit label and context [19,32]. The artifact call is TLS-Exporter with the RFC 5705 private-use label EXPERIMENTAL-ReduLink-v1, a canonical 32-byte context, and a 32-byte output. A nonexperimental deployment must register its exporter label [32]. The context is SHA-256 over a versioned, length-prefixed ALPN, scope, and endpoint-independent connection context. The latter is derived from the ALPN and an authenticated application-session identifier. aioquic 1.3.0 does not expose the required exporter through its public API, so the artifact supplies a fresh random exporter surrogate and per-run shared session identifier. Client and server invoke the connection-context derivation separately and reject disagreement. This validates context agreement but remains short of live TLS-exporter integration [27].

The ReduLink HMAC is a defensive context commitment. It detects a valid record replayed into the wrong epoch, scope, connection, direction, stream context, or reconstructed offset, and it prevents stale or corrupted dictionary bytes from being accepted as the referenced chunk. It is not an independent barrier against a network attacker already blocked by QUIC, and it does not protect against compromise of the endpoint process or exporter-derived key. Control messages such as HELLO, MISSING, and FINISH are protected by QUIC/TLS but do not carry the ReduLink record HMAC.

4.2 Key and record separation

The key schedule uses HKDF and HMAC as standardized in RFC 5869 and RFC 2104 [20,21]. Its information field is a canonical length-prefixed encoding of protocol label, ALPN, scope, connection context, stream context, direction, and fixed-width epoch. Length prefixes eliminate delimiter collisions. Tests include a concrete pair of contexts that collided under the previous delimiter-joined encoding and now derive distinct keys. Public vectors fix the byte encoding, connection context, exporter context, key-information field, and derived record secret; independently written transcript and HKDF code reproduces each value.

The keyed chunk-identifier transcript contains a versioned domain label, unsigned 64-bit epoch, length-prefixed UTF-8 scope, and 32-byte chunk SHA-256. The frame transcript contains its own versioned domain label, one-byte kind, unsigned 64-bit epoch, length-prefixed scope, unsigned 64-bit stream context and offset, 16-byte identifier, unsigned 32-bit length, unsigned 64-bit nonce, and 32-byte payload SHA-256, all in network byte order. HMAC-SHA-256 over each transcript is truncated to 128 bits. These fixed layouts replace the earlier Python JSON serialization.

HMAC security is analyzed under pseudorandom-function assumptions [20,29]. For an idealized 128-bit record tag, q independent online guesses succeed with probability at most approximately q / 2^128. Separately, n idealized 128-bit keyed identifiers collide with probability approximately n(n-1) / 2^129. A final SHA-256 over the declared complete output prevents silent acceptance of a wrong reconstruction unless that digest check also fails. The artifact does not prove guess independence, collision resistance, or protocol composition, and these statements do not replace the security analysis of QUIC/TLS.

4.3 Fail-closed receive invariants

Table 3. Receiver invariants and failure behavior.

Invariant	Check before state mutation	Failure
Context	Epoch, scope, actual stream context, direction-derived key	Reject record
Order	Expected sequence and reconstructed offset	Reject record
Replay	Bounded nonce window	Reject record
FULL integrity	Payload length, keyed identifier, record tag	Reject record
REF integrity	Dictionary presence, length, recomputed keyed identifier, record tag	Queue semantic miss or reject corruption
Expansion	Per-record chunk bound, declared input length, global byte quota	Reject transfer
Completion	Complete sequence set, exact length, final SHA-256	Reject FINISH

Privacy still depends on policy. A sender that reveals whether a receiver has a chunk can create a content-existence oracle, and response length can reveal reuse. The protocol does not solve this general deduplication side channel [23,24]. Partitioned dictionaries, authorization, padding, minimum object sizes, and rate limits are deployment requirements, not optional performance tuning.

5. Implementation

The artifact is written in Python and pins aioquic 1.3.0 and python-zstandard 0.25.0 with libzstd 1.5.7 [25-28,31]. The native experiment creates a server-authenticated localhost QUIC connection, reads the actual application stream ID at both endpoints, derives a per-stream secret from fresh per-run inputs, serializes binary messages, forces receiver dictionary misses, sends batched repair, and verifies exact reconstruction. Server certificate generation and diagnostic timing remain implementation details; no timing result is used in this paper.

The dictionary implementation uses OrderedDict-based true LRU. A regression test constructs an access pattern in which a REF hit must refresh recency to prevent a later useful entry from eviction. Protocol-vector tests reproduce the binary MAC transcripts and key schedule with separate encoding and HKDF code before checking the production functions. Other tests reject delimiter-ambiguous contexts, invalid 62-bit IDs and offsets, wrong HELLO lengths, reconstruction above quota, wrong scope and stream context, replayed nonces, corrupted dictionary bytes, wrong repair metadata, and truncated or trailing wire messages.

The object suite serializes an ordered mapping of relative name, object length, and contents. Empty objects and removed or added paths are preserved in the expected mapping. Secure object headers include a tag over index, name, and length. Every secure frame completes a binary encode/decode round trip before result acceptance. The logical object-context identifier is the low 62 bits of SHA-256(name), and the suite rejects a collision within a transfer. These object experiments use a documented deterministic public artifact key. They test byte serialization, tag verification, and reconstruction exactness, not key secrecy or live TLS key export.

6. Experimental Methodology

6.1 Evidence layers

The evaluation separates representation evidence from transport diagnostics. Input bytes are the exact bytes to reconstruct. Protocol-stream bytes are the FULL, REF, HELLO, END_ROUND, MISSING, repair, and FINISH messages written to QUIC streams. The final STATS response is reported separately and excluded. UDP proxy observations are retained in the artifact but are not mixed with stream bytes. No packet-level or link-level multiplier is claimed.

Table 4. Evidence hierarchy and supported inference.

Evidence	What is controlled	Supported inference	Not supported
Hash-pinned public releases	Exact archive hashes and ordered object extraction	Object reuse and baseline byte counts	Production traffic prevalence
PyPI wheel pairs	Exact wheel hashes and member names/content	Package-object reuse on the four recorded pairs	All package ecosystems
Native aioquic mapping	Actual encrypted stream, binary messages, exact digest	Protocol-stream bytes and state behavior	WAN latency or throughput
Capacity, chunk, and miss sweeps	One deterministic byte workload per point	Sensitivity to state, chunk size, and repair	Population statistics
Security tests	Malformed fields and state transitions	Implemented fail-closed behavior	Formal verification or endpoint-compromise security

6.2 Workloads

Public releases retrieved from three upstream projects are hash pinned: Click 8.1.7 to 8.1.8, Redis 7.2.4 to 7.2.5, and Nginx 1.25.3 to 1.25.4. Each pair is evaluated in two representations. The raw-tree serialization tests whether a generic byte stream retains aligned chunks. The named-object serialization resets chunk boundaries at each file and models a registry or object service. These are real public bytes but not production traces.

Four recorded PyPI wheel pairs provide a second object corpus: rich 13.7.0 to 13.7.1, Jinja2 3.1.3 to 3.1.4, Click 8.1.6 to 8.1.7, and Werkzeug 3.0.1 to 3.0.2. Both wheel hashes are stored in the result. A 512 KiB layer-like case is constructed by reblocking public Redis release bytes and changing five aligned blocks. It is an author-constructed positive fixture, not a registry trace. Native QUIC cases also include an independently compressed negative control.

6.3 Baselines and exactness

The 4 KiB token baseline uses a 32-byte content token for an existing chunk and a 32-byte literal header for a new chunk. It is not called rsync. The whole-object CAS baseline sends a relative-name and length header plus a 32-byte object digest; an unchanged object is a digest reference and a new object includes its full bytes. The decoder reconstructs the exact ordered named-object mapping.

The zstd baseline supplies the entire prior serialized object stream as a raw-content dictionary to libzstd 1.5.7 at level 3, pins window_log to 21, and enables a frame checksum. It then decompresses with the same dictionary and compares exact bytes and SHA-256. Raw-content dictionaries are supported by the Zstandard format and implementation documentation [25,26]. A window_log 24 sensitivity run is recorded for every pair. This baseline omits RFC 9842's HTTP negotiation, dictionary identity headers, origin rules, and dcz framing. The gzip baseline uses level 6 with mtime set to zero, records Python and compile-time/runtime zlib versions, decompresses its output, and checks both bytes and SHA-256.

The raw-tree ReduLink baseline serializes HELLO, every FRAME, END_ROUND, an empty MISSING batch, and FINISH. It then parses every length prefix and binary message, reconstructs from the decoded SecureFrame values rather than the original in-memory objects, and compares the complete bytes and SHA-256. This check makes its quoted wire-byte count and exactness claim one executable profile.

Real rsync 3.2.7 runs five times per pair with recursive, symlink, checksum, delete, no-whole-file, fixed-checksum-seed, and stats options. Its reported denominator is the observed median of rsync's total bytes sent plus received, with every run retained in the result. After each transfer, a canonical manifest hashes every relative path, entry type, file content or symlink target, and length. Equal total byte counts are insufficient. A row is accepted only when the receiver manifest exactly equals the new source tree.

Except for rsync, the load-bearing results are deterministic functions of fixed input bytes, fixed encodings, and recorded parameters. They are exhaustive measurements of those artifacts rather than estimates of a sampled population, so confidence intervals would not be meaningful. Rsync is repeated five times because its observed protocol total varies slightly; the paper reports the median and retains every run. Generalization is assessed with distinct corpora, explicit negative controls, and parameter sweeps rather than inferential statistics.

7. Results

7.1 Named public objects

Figure 2 and Table 5 show that no method dominates. ReduLink's binary HMAC-frame profile under the deterministic artifact key reduces bytes on all three named-object pairs, reaching 1.81x for Click, 7.48x for Redis, and 3.92x for Nginx. The simpler 4 KiB token baseline is consistently smaller because it has less metadata. Whole-object CAS remains competitive when many complete files are unchanged. The zstd prior-stream dictionary is strongest on Click and substantially stronger than ReduLink on Nginx, but it is weaker than ReduLink on Redis. gzip operates without prior state and is strongest on Nginx among the non-dictionary rows. The zstd window sensitivity leaves Click unchanged at 58.11x, changes Redis from 4.62x to 4.60x, and improves Nginx from 6.01x to 6.95x at window_log 24. The headline table retains the pinned window_log 21 results; the qualitative comparison is unchanged.

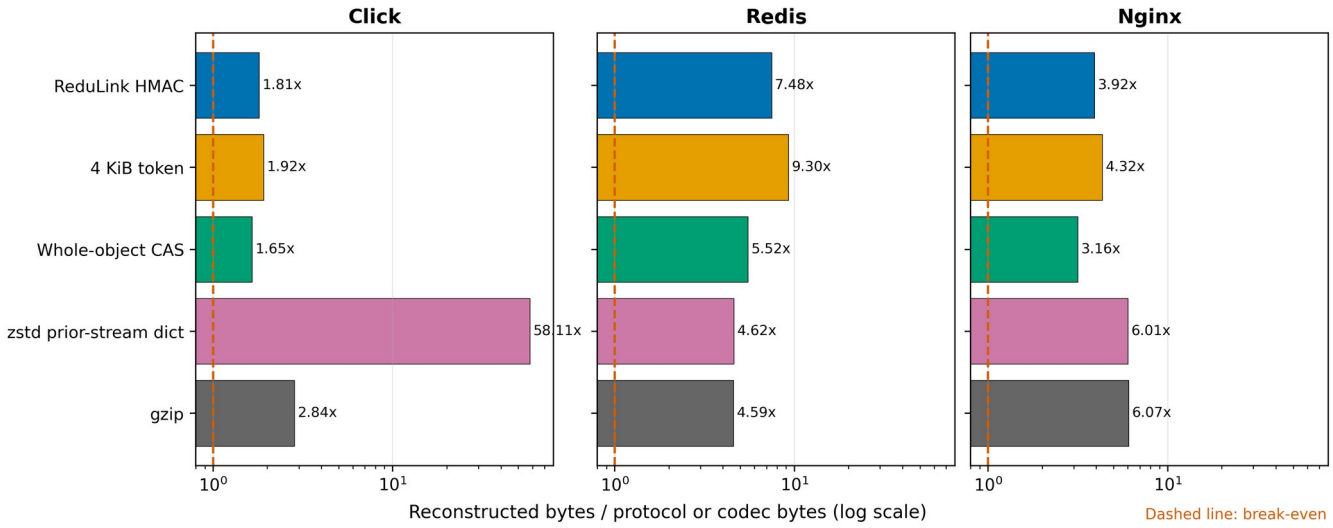

Figure 2. Public named-object pairs at each method's stated byte layer; zstd uses pinned window_log 21. The log axis and break-even line show gains and negative comparisons.

Table 5. Public named-object results. Every stateful method shown completed exact reconstruction.

Pair	Input bytes	ReduLink HMAC	4 KiB token	Whole-object CAS	zstd w21	gzip
Click	947,826	1.81x	1.92x	1.65x	58.11x	2.84x
Redis	15,467,095	7.48x	9.30x	5.52x	4.62x	4.59x
Nginx	7,357,392	3.92x	4.32x	3.16x	6.01x	6.07x

7.2 Fixed-chunk sensitivity

The 4 KiB choice is not universally optimal. With byte-equivalent 64 MiB dictionaries, Click peaks at 1.81x at 4 KiB, Nginx improves from 3.92x at 4 KiB to 4.01x at 8 KiB, and Redis improves from 7.48x at 4 KiB to 8.20x at 16 KiB. Thus, 4 KiB is a transparent reference point and is near the local optimum for Click, but deployments should tune chunk size against object structure and metadata cost.

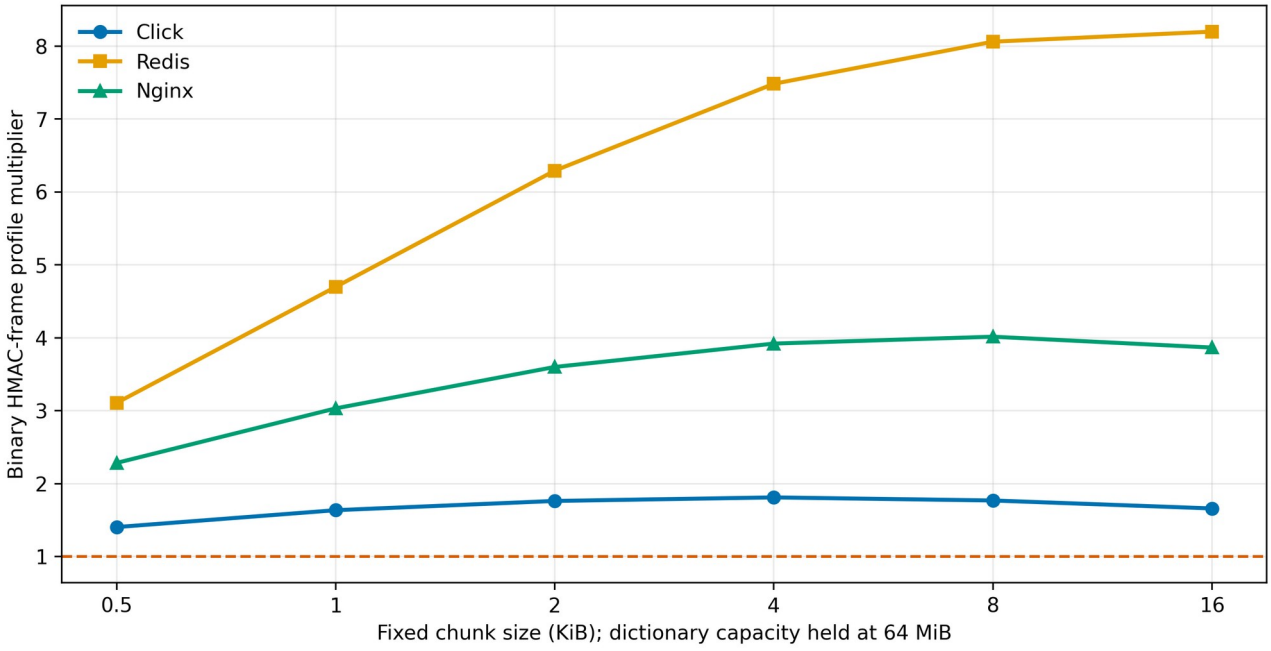

Figure 3. Named-object sensitivity to fixed chunk size with byte-equivalent 64 MiB dictionaries. All points reconstruct exactly.

7.3 Raw source trees and real rsync

Object alignment is a deployment assumption, not a property of arbitrary streams. When the same releases are serialized as raw trees with file-boundary markers, fixed-chunk ReduLink's complete binary profile is below break-even at 0.97x to 0.98x. Real rsync reaches 6.67x, 103.17x, and 56.85x while reconstructing exact tree manifests. The ReduLink multiplier uses the canonical serialized-tree byte length, whereas rsync uses regular-file payload bytes divided by the median total bytes sent plus received. The table exposes both numerators because these cross-tool values describe the same release pairs but are not identical accounting layers. The result supports choosing rsync rather than ReduLink for file-tree synchronization.

Table 6. Raw source-tree transfer is a negative case for ReduLink and a strong case for rsync; numerators are shown explicitly.

Pair	Binary numerator	ReduLink binary	rsync numerator	rsync total	Exact tree
Click	953,008	0.97x	947,826	6.67x	yes
Redis	15,533,278	0.98x	15,467,095	103.17x	yes
Nginx	7,377,697	0.98x	7,357,392	56.85x	yes

7.4 PyPI object pairs

The four wheel pairs broaden the object result without changing its interpretation. The binary HMAC-frame ReduLink multiplier ranges from 1.06x to 9.26x. Whole-object CAS ranges from 1.07x to 10.87x and beats ReduLink on rich while trailing it on Click and Werkzeug. gzip wins on the low-reuse Jinja2 pair. These rows show why unchanged-object fraction and intra-object compressibility must be reported alongside a ReduLink result.

Table 7. Hash-pinned PyPI wheel member sequences.

Pair	Unchanged/new	Input bytes	ReduLink HMAC	Whole-object CAS	gzip
rich 13.7.0 to 13.7.1	77/83	947,654	9.26x	10.87x	4.42x
jinja2 3.1.3 to 3.1.4	8/31	491,214	1.06x	1.07x	4.02x
click 8.1.6 to 8.1.7	14/22	353,767	8.75x	7.61x	3.97x
werkzeug 3.0.1 to 3.0.2	51/72	762,091	2.81x	2.28x	3.95x

7.5 Native QUIC accounting

The zero-loss native comparison sends the same 98,304-byte update as raw stream data and as ReduLink records. Raw QUIC writes 98,304 protocol-stream bytes. ReduLink writes 30,530 forward bytes and 321 reverse repair/control bytes, for 30,851 total and 3.19x. Both diagnostic STATS responses are excluded. Their serialized size is deliberately omitted because it is not protocol evidence and can vary with diagnostic JSON formatting. This result is same-layer accounting, not a congestion fairness experiment.

Table 8. Same-layer zero-loss native QUIC stream accounting.

Method	Input bytes	Forward protocol	Reverse repair/control	Protocol total	Multiplier
Raw QUIC	98,304	98,304	0	98,304	1.00x
ReduLink	98,304	30,530	321	30,851	3.19x

The workload controls confirm conditionality. The deterministic warm update reaches 3.19x. The independent compressed control has no semantic misses and expands to 0.91x because record metadata exceeds any reuse. The author-constructed Redis-derived layer fixture reaches 3.75x after 72 semantic repairs. All three SHA-256 checks pass.

Table 9. Native QUIC positive and negative workload controls.

Case	Input bytes	Protocol bytes	Multiplier	Misses	Exact
demo-positive	98,304	30,851	3.19x	13	yes
independent-compressed-negative	262,242	288,273	0.91x	0	yes
external-positive-redis-layered	524,288	139,634	3.75x	72	yes

7.6 Matched endpoint dictionary capacity

Capacity is a first-order condition. This sweep gives sender and receiver the same true-LRU budget and disables artificial receiver thinning, so it isolates the effect of matched endpoint working-set capacity. The multiplier reaches 10.06x at 8 MiB with 8,192 chunks. At 16 MiB, the same limit causes sequential insertion to evict useful future chunks before they are

referenced; all initial records become FULL and the multiplier falls to 0.91x. Raising both endpoint budgets to 24,576 chunks retains the warm set and recovers 10.10x. Exact reconstruction passes in every row.

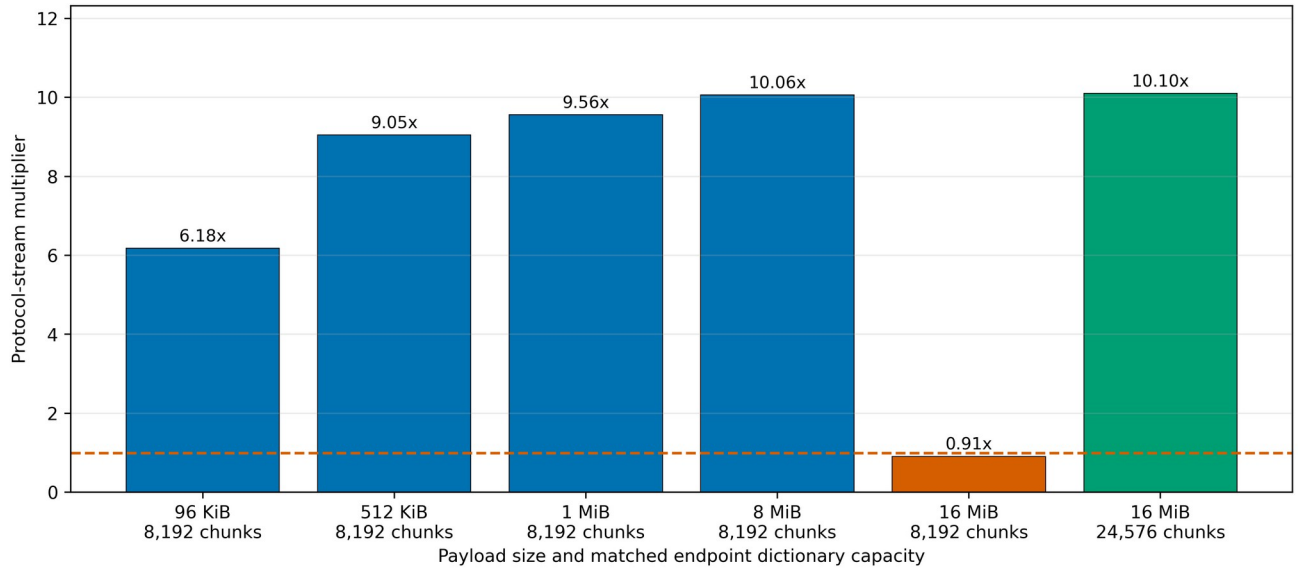

Figure 4. Native QUIC scaling with bounded true LRU. The paired 16 MiB rows isolate matched sender and receiver capacity.

Table 10. Native QUIC capacity sweep. Diagnostic STATS bytes are excluded.

Input bytes	Chunks per endpoint	Protocol bytes	Misses	Multiplier	Exact
98,304	8,192	15,914	0	6.18x	yes
524,288	8,192	57,930	0	9.05x	yes
1,048,576	8,192	109,642	0	9.56x	yes
8,388,608	8,192	833,610	0	10.06x	yes
16,777,216	8,192	18,432,074	0	0.91x	yes
16,777,216	24,576	1,661,002	0	10.10x	yes

7.7 Semantic miss cost

The miss sweep keeps input bytes and initial REF count fixed while thinning receiver state. With no thinning, all 90 references resolve and the protocol multiplier is 6.18x. At 48.9% misses, 44 batched FULL repairs raise forward bytes from 15,905 to 65,405 and reverse repair/control bytes from 9 to 1,065, reducing the multiplier to 1.48x. At the measured 100 percent-miss endpoint it is 0.82x. The discrete sweep therefore brackets break-even between 48.9 and 100 percent misses; it does not claim a more precise crossing. The curve is monotonic for this deterministic workload and makes the repair cost visible without converting it into a timing claim.

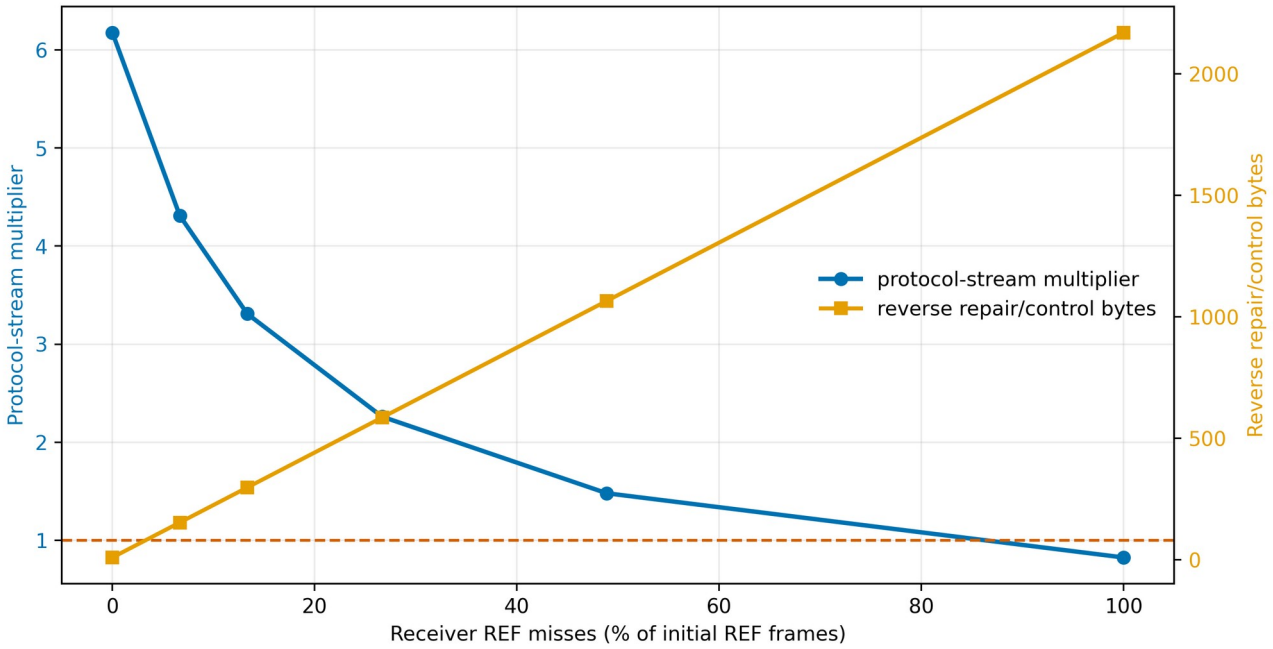

Figure 5. Native QUIC byte sensitivity to semantic reference misses. Each point is one deterministic accounting run.

Table 11. Semantic miss and repair byte sensitivity.

Miss fraction	Misses	Forward bytes	Reverse bytes	Protocol total	Multiplier
0.0%	0	15,905	9	15,914	6.18x
6.7%	6	22,655	153	22,808	4.31x
13.3%	12	29,405	297	29,702	3.31x
26.7%	24	42,905	585	43,490	2.26x
48.9%	44	65,405	1,065	66,470	1.48x
100.0%	90	117,155	2,169	119,324	0.82x

8. Discussion

8.1 Answers to the research questions

RQ1 is answered for the implemented application-stream profile. The prototype uses actual aioquic stream IDs, fixed binary key and record transcripts with public vectors, authenticated FULL/REF records, ordered offsets, bounded nonces, true LRU, a QUIC/TLS-protected declaration checked against server-configured expectations, reconstruction quota, exact completion digest, and validated batched repair. This establishes executable state behavior, not a complete production protocol. Live TLS exporter integration, client authorization, and on-wire warm-state admission remain absent.

RQ2 has a workload-dependent answer. ReduLink reduces bytes on the three public named-object pairs and four recorded wheel pairs. It fails on raw source trees where rsync is far stronger, and it expands an independent compressed control. Whole-object CAS and 4 KiB tokens often approach or exceed it with lower metadata. zstd dictionary compression can be much stronger when the complete prior stream is retained and codec-level reconstruction is acceptable. ReduLink's relevant benefit is not byte optimality. It is explicit chunk resolution with scoped record commitments and a semantic repair path.

RQ3 shows two independent break-even mechanisms. Insufficient matched endpoint capacity can cause LRU thrashing even when both endpoints initially hold useful bytes. Semantic misses add both reverse control and forward literals. A deployment must therefore estimate working-set size and miss probability before enabling ReduLink. A sender should fall back to raw or compressed transfer when predicted protocol bytes approach the input size.

8.2 Choosing the appropriate mechanism

Table 12. Deployment decision guide.

Workload and state	Preferred mechanism	Reason
HTTP response with a prior response	RFC 9842 CDT	HTTP dictionary negotiation and scope
Related named file tree	rsync or file delta	Rolling matches and file metadata
Exact prior byte stream retained	zstd dictionary or delta codec	Compact codec output

Workload and state	Preferred mechanism	Reason
Mostly unchanged objects	Whole-object CAS	Low reference overhead
Managed QUIC object stream needing chunk repair	ReduLink candidate	Bounded records and batched repair
No warm reuse or high entropy	Raw or ordinary compression	References cannot amortize

This positioning also clarifies security language. RFC 9842 provides hashed dictionary identity, secure-context requirements, origin and readability policy, and response failure rules [13]. ReduLink's HMAC should not be described as stronger network integrity. It is a different internal state-binding primitive at chunk granularity.

9. Limitations and Threats to Validity

Several limits constrain the contribution. First, endpoint redundancy elimination, content-addressed chunks, and cache-miss repair are established ideas [4-10,30]. The novelty is the bounded QUIC mapping and its evaluation, not reference substitution itself. Second, the native code uses an exporter surrogate and server-only TLS certificate authentication. Application-level client authorization is assumed, not implemented. Third, warm state is preprovisioned; discovery, admission, revocation, and synchronization protocols are outside scope. Fourth, repair is batched and reconstructed output is buffered until FINISH, so the paper makes no incremental delivery or time-to-first-byte claim. Reconstructed bytes are not coupled to QUIC flow-control credit; only encoded stream bytes consume transport credit in the prototype.

Fifth, 0-RTT behavior is undefined and disabled, and connection-migration dictionary policy is not implemented. A production design must bind state to the accepted TLS session and define whether migration retains or invalidates that state. Sixth, all live QUIC experiments are single-host aioquic runs. They validate encrypted stream behavior and byte counts, not WAN latency, congestion fairness, kernel queueing, CPU scalability, or interoperability. Historical timing files are excluded from the submission evidence. Seventh, public releases and wheels are reproducible artifacts rather than sampled production traces, and the Redis layer fixture is explicitly author constructed. Eighth, the main chunker is fixed-size. The included rolling-hash CDC prototype is not presented as FastCDC or used for headline claims [22].

The 128-bit tag argument relies on standard assumptions and is not machine checked. The tests demonstrate implementation behavior for enumerated malformed inputs; they are not a proof of memory safety or endpoint security. Python performance is not representative of an optimized implementation. Dictionary content can create compression and deduplication side channels even when every record is authentic [13,23,24].

10. Reproducibility and Artifact Scope

The repository records source, manifests, result CSV and JSON files, figure scripts, the manuscript builder, dependency locks, and tests [28]. SOURCE_COMMIT.txt identifies the code revision used to generate the evidence as ba16d4173bec880ed8d4f653fc531663936fc0b1. Result-producing scripts report directional protocol bytes and exact reconstruction. The CI smoke workflow writes generated outputs to temporary paths and fails if a smoke command rewrites committed evidence. In CI, benchmark commands regenerate deterministic CSV/JSON outputs and the evidence checker compares their load-bearing fields with committed rows. In its default mode, the checker rebuilds figures and the normalized DOCX package in temporary paths and compares them with the committed submission artifact. It also extracts the fixed-layout PDF and requires the recorded source revision, headline capacity and miss values, title, and AI-use declaration. It verifies that the recorded source revision is a repository ancestor. Rsync totals use a documented tolerance because the tool's control bytes vary slightly across repeated runs.

The repository README and evidence map give one command for each result family. The full validator executes the isolated test suite, while the figure and manuscript builders regenerate every submission graphic and the editable paper from committed evidence.

External archives and wheels are not redistributed when their upstream distribution is sufficient. Their versions, URLs or package specifications, sizes, and SHA-256 digests are stored in manifests or result files. Historical timing and path-emulation outputs are outside the submission evidence set.

11. Conclusion

ReduLink is a bounded representation profile for a specific endpoint condition: a QUIC receiver already holds authorized warm object state, and the application values explicit chunk resolution and fail-closed state handling. The implementation binds records to canonical connection and stream context, enforces true-LRU and reconstruction quotas, validates batched repair, and separates forward protocol, reverse control, and diagnostic bytes. Public object pairs show useful but non-universal savings. Direct baselines show where whole-object CAS, gzip, zstd dictionaries, or rsync are preferable. Capacity overflow and miss sweeps expose the two main break-even risks. The resulting claim is narrower than a general QUIC accelerator, but it is supported by exact reconstruction, same-layer accounting, negative controls, chunk-size and state sensitivity, and a reproducible artifact.

Data and Code Availability

Source code, manifests, generated evidence, figures, and validation scripts are available at https://github.com/pinkysworld/redulink-deduplex-quic [28]. The repository records the exact source revision associated with the submitted evidence.

Declaration of Generative AI and AI-Assisted Technologies

During preparation, the author used OpenAI ChatGPT and Codex to support language editing, code review, benchmark validation, and document production. The author reviewed and verified the resulting text, code, data, references, and analyses and accepts full responsibility for the work.

References

[1] N. T. Spring and D. Wetherall, "A protocol-independent technique for eliminating redundant network traffic," Proc. ACM SIGCOMM, pp. 87-95, 2000. doi:10.1145/347059.347408.

[2] A. Anand, V. Sekar, and A. Akella, "SmartRE: An architecture for coordinated network-wide redundancy elimination," Proc. ACM SIGCOMM, pp. 87-98, 2009. doi:10.1145/1592568.1592580.

[3] A. Anand, A. Gupta, A. Akella, S. Seshan, and S. Shenker, "Redundancy in network traffic: Findings and implications," Proc. ACM SIGMETRICS, pp. 37-48, 2009. doi:10.1145/1555349.1555355.

[4] B. Aggarwal et al., "EndRE: An end-system redundancy elimination service for enterprises," Proc. 7th USENIX NSDI, 2010. https://www.usenix.org/conference/nsdi10-0/endre-end-system-redundancy-elimination-service-enterprises.

[5] N. Tolia, M. Kaminsky, D. G. Andersen, and S. Patil, "An architecture for Internet data transfer," Proc. 3rd USENIX NSDI, 2006. https://www.usenix.org/events/nsdi06/tech/full_papers/tolia/tolia_html.

[6] E. Zohar, I. Cidon, and O. Mokryn, "PACK: Prediction-based cloud bandwidth and cost reduction system," IEEE/ACM Trans. Netw., vol. 22, no. 1, pp. 39-51, 2014. doi:10.1109/TNET.2013.2240010.

[7] L. Yu, H. Shen, K. Sapra, L. Ye, and Z. Cai, "CoRE: Cooperative end-to-end traffic redundancy elimination for reducing cloud bandwidth cost," IEEE Trans. Parallel Distrib. Syst., vol. 28, no. 2, pp. 446-461, 2017. doi:10.1109/TPDS.2016.2578928.

[8] A. Tridgell and P. Mackerras, "The rsync algorithm," Australian National University, Tech. Rep. TR-CS-96-05, 1996. http://hdl.handle.net/1885/40765.

[9] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," Proc. ACM SOSP, pp. 174-187, 2001. doi:10.1145/502034.502052.

[10] P. Kulkarni, F. Douglis, J. LaVoie, and J. M. Tracey, "Redundancy elimination within large collections of files," Proc. USENIX ATC, pp. 59-72, 2004. https://www.usenix.org/conference/2004-usenix-annual-technical-conference/redundancy-elimination-within-large-collections.

[11] J. Mogul et al., "Delta encoding in HTTP," RFC 3229, IETF, 2002. doi:10.17487/RFC3229.

[12] D. Korn, J. MacDonald, J. Mogul, and K. Vo, "The VCDIFF generic differencing and compression data format," RFC 3284, IETF, 2002. doi:10.17487/RFC3284.

[13] P. Meenan and Y. Weiss, "Compression Dictionary Transport," RFC 9842, IETF, 2025. doi:10.17487/RFC9842.

[14] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, 2021. doi:10.17487/RFC9000.

[15] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, IETF, 2021. doi:10.17487/RFC9001.

[16] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," RFC 9002, IETF, 2021. doi:10.17487/RFC9002.

[17] M. Kuehlewind and B. Trammell, "Applicability of the QUIC transport protocol," RFC 9308, IETF, 2022. doi:10.17487/RFC9308.

[18] B. Trammell et al., "Manageability of the QUIC transport protocol," RFC 9312, IETF, 2022. doi:10.17487/RFC9312.

[19] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, sec. 7.5, IETF, 2026. doi:10.17487/RFC9846.

[20] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, IETF, 1997. doi:10.17487/RFC2104.

[21] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, 2010. doi:10.17487/RFC5869.

[22] W. Xia et al., "The design of fast content-defined chunking for data deduplication based storage systems," IEEE Trans. Parallel Distrib. Syst., vol. 31, no. 9, pp. 2017-2031, 2020. doi:10.1109/TPDS.2020.2984632.

[23] D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Side channels in cloud services: Deduplication in cloud storage," IEEE Security & Privacy, vol. 8, no. 6, pp. 40-47, 2010. doi:10.1109/MSP.2010.187.

[24] M. Bellare, S. Keelveedhi, and T. Ristenpart, "DupLESS: Server-aided encryption for deduplicated storage," Proc. USENIX Security, pp. 179-194, 2013. https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/bellare.

[25] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," RFC 8878, IETF, 2021. doi:10.17487/RFC8878.

[26] Zstandard project, "Dictionary format and raw content dictionaries," software documentation, version 1.5.7, accessed July 15, 2026. https://github.com/facebook/zstd/blob/v1.5.7/doc/zstd_compression_format.md.

[27] aioquic contributors, "aioquic: QUIC and HTTP/3 implementation in Python," version 1.3.0, software, released October 11, 2025, accessed July 15, 2026. https://pypi.org/project/aioquic/1.3.0/.

[28] M. Nguyen, "ReduLink artifact and reproducibility package," source repository, 2026. https://github.com/pinkysworld/redulink-deduplex-quic.

[29] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," Advances in Cryptology, CRYPTO 1996, pp. 1-15. doi:10.1007/3-540-68697-5_1.

[30] Tentackle project, "TRIP over QUIC: the tentackle-quic module," implementation documentation, accessed July 15, 2026. https://tentackle.org/quic/.

[31] python-zstandard contributors, "zstandard Python bindings," version 0.25.0, software, accessed July 15, 2026. https://pypi.org/project/zstandard/0.25.0/.

[32] E. Rescorla, "Keying Material Exporters for Transport Layer Security (TLS)," RFC 5705, secs. 3-4, IETF, 2010. doi:10.17487/RFC5705.